AutorizaJá

FASE 2 · MACRO ENTREGA 2

Artefato 05

Estrutura Técnica do Projeto

Repositório, tecnologias, dependências e padrões de desenvolvimento

Projeto	AutorizaJá: acompanhamento de autorizações em planos de saúde
Aluno	Tony Tomaz
Instituição	UNIFACISA
Entrega	12 de novembro de 2026 · versão 1.0

1. Repositório

Item	Valor
Plataforma	GitHub
URL	**https://github.com/Thezntt/Autoriza-ja**
Organização	Monorepo (frontends, API e documentação no mesmo repositório)
Gerenciador de pacotes	pnpm com *workspaces*
Licença	MIT

O monorepo foi escolhido porque o projeto tem um único desenvolvedor e os três aplicativos compartilham tipos TypeScript (contratos da API). Assim, uma mudança de contrato é feita e revisada em um único pull request.

2. Estrutura de pastas

```
autorizaja/
├── apps/
│   ├── api/                    # Backend NestJS (monólito modular)
│   │   ├── prisma/
│   │   │   ├── schema.prisma   # Modelo de dados (artefato 03)
│   │   │   ├── migrations/
│   │   │   └── seed.ts         # Dados fictícios para desenvolvimento
│   │   ├── src/
│   │   │   ├── main.ts
│   │   │   ├── app.module.ts
│   │   │   ├── common/         # guards, filtros de erro, interceptors, logger
│   │   │   ├── config/         # leitura e validação de variáveis de ambiente
│   │   │   └── modules/
│   │   │       ├── auth/
│   │   │       ├── beneficiarios/
│   │   │       ├── pedidos/
│   │   │       │   ├── pedidos.controller.ts
│   │   │       │   ├── pedidos.service.ts
│   │   │       │   ├── pedidos.repository.ts
│   │   │       │   ├── domain/   # entidades, máquina de estados, eventos
│   │   │       │   └── dto/
│   │   │       ├── prazos/
│   │   │       ├── documentos/
│   │   │       ├── regulacao/
│   │   │       ├── notificacoes/
│   │   │       ├── integracao/   # ConectorERP + AdapterRest + AdapterVisaoBanco
│   │   │       ├── relatorios/
│   │   │       └── auditoria/
│   │   ├── src/worker.ts         # processo separado para filas BullMQ
│   │   └── test/                 # testes e2e
│   ├── portal-beneficiario/      # React + Vite (PWA)
│   │   └── src/{pages,components,hooks,services,styles}
│   └── painel-operadora/         # React + Vite (SPA)
│       └── src/{pages,components,hooks,services,styles}
├── packages/
│   ├── shared-types/             # tipos e enums compartilhados (StatusPedido etc.)
│   └── ui/                       # componentes visuais comuns (selo de status, barra de prazo)
├── docs/
│   ├── fase-1/                   # artefatos 01 a 05 da Macro Entrega 1
│   ├── fase-2/                   # artefatos 01 a 05 da Macro Entrega 2
│   ├── diagramas/                # fontes Mermaid (.mmd)
│   └── adr/                      # registros de decisão arquitetural
├── infra/
│   ├── docker-compose.yml        # postgres, redis, minio, mailpit
│   └── nginx/
├── .github/
│   ├── workflows/ci.yml          # lint, testes e build a cada PR
│   └── pull_request_template.md
├── .env.example
├── package.json
├── pnpm-workspace.yaml
└── README.md
```

3. Tecnologias e dependências

3.1 Backend (`apps/api`)

Dependência	Versão	Uso
Node.js	20 LTS	Ambiente de execução
TypeScript	5.x	Linguagem
@nestjs/core, @nestjs/common	10.x	Framework modular da API
@nestjs/jwt, @nestjs/passport	10.x	Autenticação JWT
prisma, @prisma/client	5.x	ORM e migrações do PostgreSQL
bullmq	5.x	Filas de tarefas sobre Redis
ioredis	5.x	Cliente Redis (OTP, cache)
@aws-sdk/client-s3	3.x	Armazenamento de anexos (S3/MinIO)
argon2	0.x	Hash de senha
class-validator, class-transformer	0.14.x	Validação de entrada
helmet, @nestjs/throttler	-	Cabeçalhos de segurança e rate limit
pdfkit	0.15.x	Geração da resposta formal em PDF
nodemailer	6.x	Envio de e-mail
date-fns	3.x	Cálculo de datas e dias úteis
pino	9.x	Logs estruturados em JSON

3.2 Frontends (`apps/portal-beneficiario`, `apps/painel-operadora`)

Dependência	Versão	Uso
React	18.x	Interface
Vite	5.x	Build e servidor de desenvolvimento
Tailwind CSS	3.x	Estilos
React Router	6.x	Rotas
TanStack Query	5.x	Chamadas à API, cache e atualização de dados
TanStack Table	8.x	Tabela da fila de regulação (painel)
React Hook Form + Zod	-	Formulários e validação
vite-plugin-pwa	0.x	Manifesto e *service worker* (portal)
Recharts	2.x	Gráficos do dashboard (painel)

3.3 Infraestrutura e ferramentas

Ferramenta	Uso
Docker + Docker Compose	Ambiente local com PostgreSQL 16, Redis 7, MinIO e Mailpit
GitHub Actions	Integração contínua: instalar, lint, testes e build
ESLint + Prettier	Padrão de código e formatação automática
Husky + lint-staged + commitlint	Verificações antes de cada commit
Jest (API) e Vitest (front)	Testes unitários
Supertest	Testes de integração dos endpoints
Playwright	Testes ponta a ponta dos fluxos principais
OWASP ZAP	Varredura de segurança antes da release

4. Como executar localmente

```
git clone https://github.com/Thezntt/Autoriza-ja.git
cd Autoriza-ja
cp .env.example .env
docker compose -f infra/docker-compose.yml up -d   # postgres, redis, minio, mailpit
pnpm install
pnpm --filter api prisma migrate dev
pnpm --filter api prisma db seed                   # dados fictícios
pnpm dev                                           # sobe API, worker e os dois frontends
```

Serviço	Endereço local
API	http://localhost:3000 (documentação Swagger em `/docs`)
Portal do Beneficiário	http://localhost:5173
Painel da Operadora	http://localhost:5174
Caixa de e-mails de teste (Mailpit)	http://localhost:8025

Variáveis de ambiente (`.env.example`): `DATABASE_URL`, `REDIS_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `SMTP_HOST`, `WHATSAPP_TOKEN`, `ERP_ADAPTER` (`rest` ou `visao`), `ERP_URL`. Segredos reais nunca são versionados.

5. Padrões de desenvolvimento

5.1 Fluxo de branches (GitHub Flow simplificado)

- `main`: sempre estável e implantável; protegida (só recebe PR aprovado com CI verde).

- `feat/US05-envio-documento`, `fix/prazo-feriado`, `docs/arquitetura`: uma branch por item do backlog.
- Pull request obrigatório, com checklist do template e referência à história (US) ou item (PB).

5.2 Mensagens de commit (Conventional Commits)

```
feat(documentos): permite envio de foto pelo portal (US05)
fix(prazos): considera feriado municipal no cálculo de dias úteis
docs(arquitetura): adiciona ADR-07 sobre prazos parametrizáveis
test(regulacao): cobre validação de negativa sem fundamentação
```

5.3 Padrões de código

Tema	Padrão
Nomes	Código em inglês técnico para estruturas (`controller`, `service`) e termos de domínio em português (`pedido`, `parecer`), igual ao vocabulário do negócio
Arquitetura	Controlador só recebe e responde; regra de negócio fica em `service` / `domain`; acesso a dados só em `repository`
Erros	Exceções de domínio convertidas em respostas HTTP por um filtro global, no formato `{ codigo, mensagem, campos }`
API	REST, recursos no plural (`/pedidos/{id}/documentos`), versionamento por prefixo `/v1`, documentação OpenAPI
Testes	Toda regra de negócio (RN) tem teste unitário; fluxos críticos (US05, US12) têm teste ponta a ponta
Segurança	Nenhum dado pessoal em log; validação de entrada em todos os DTOs; dependências verificadas com `pnpm audit`
Revisão	Código gerado por IA é revisado como qualquer outro (ver Plano de IA)

5.4 Pipeline de integração contínua

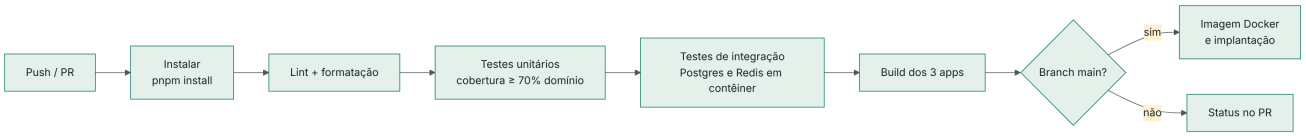


Figura 1: Pipeline do GitHub Actions executada a cada pull request

5.5 Definition of Done técnica

- Lint e testes passando na pipeline.
- PR revisado e vinculado à história.
- Documentação atualizada quando muda contrato da API ou o modelo de dados.
- Migração de banco versionada (nunca alterar banco manualmente).